problem isn't something the C++ Thread Library can help you with; it's up to you as programmers to lock the right mutex to protect your data. On the upside, you have a guideline to follow, which will help you in these cases: *Don't pass pointers and references to protected data outside the scope of the lock, whether by returning them from a function, storing them in externally visible memory, or passing them as arguments to user-supplied functions.*

Although this is a common mistake when trying to use mutexes to protect shared data, it's far from the only potential pitfall. As you'll see in the next section, it's still possible to have race conditions, even when data is protected with a mutex.

3.2.3. Spotting race conditions inherent in interfaces

Just because you're using a mutex or other mechanism to protect shared data, you're not necessarily protected from race conditions; you still have to ensure that the appropriate data is protected. Consider the doubly linked list example again. In order for a thread to safely delete a node, you need to ensure that you're preventing concurrent accesses to three nodes: the node being deleted and the nodes on either side. If you protected accesses to the pointers of each node individually, you'd be no better off than with code that used no mutexes, because the race condition could still happen—it's not the individual nodes that need protecting for the individual steps but the whole data structure, for the whole delete operation. The easiest solution in this case is to have a single mutex that protects the entire list, as in listing 3.1.

Just because individual operations on the list are safe, you're not out of the woods yet; you can still get race conditions, even with a really simple interface. Consider a stack data structure like the `std::stack` container adapter shown in listing 3.3. Aside from the constructors and `swap()`, there are only five things you can do to a `std::stack`: you can `push()` a new element onto the stack, `pop()` an element off the stack, read the `top()` element, check whether it's `empty()`, and read the number of elements—the `size()` of the stack. If you change `top()` so that it returns